

東京 2025 LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

Python-BPF: Writing eBPF programs in pure Python



東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

Problem!

- Ways to write eBPF:
  - C (extended C)
  - Rust (aya-rs)
  - Python / Lua (BCC)
  - DSLs (bpftrace)

Problem!

- Focusing on quick and easier ways
- BCC
  - Not really Python
- bpftrace
  - Not extensible

Problem!

- https://github.com/iovisor/bcc/blob/master/examples/tracing/bitehist.py

```
from __future__ import print_function
from bcc import BPF
from time import sleep

# load BPF program
b = BPF(text="""
#include <uapi/linux/ptrace.h>
#include <linux/blk-mq.h>

BPF_HISTOGRAM(dist);
BPF_HISTOGRAM(dist_linear);

int trace_req_done(struct pt_regs *ctx, struct request *req)
{
    dist.increment(bpf_log2l(req->__data_len / 1024));
    dist_linear.increment(req->__data_len / 1024);
    return 0;
}
""")

if BPF.get_kprobe_functions(b'__blk_account_io_done'):
    # __blk_account_io_done is available before kernel v6.4.
    b.attach_kprobe(event="__blk_account_io_done", fn_name="trace_req_done")
```

Python-BPF

- We are building Python-BPF to solve these problems
  - A reduced Python grammar
  - Can use existing Python dev tooling

Introduction

- BCC vs Python-BPF for the examples/tracing/hello_perf_output.py example.

```
# define BPF program
prog = """
#include <linux/sched.h>

// define output data structure in C
struct data_t {
    u32 pid;
    u64 ts;
    char comm[TASK_COMM_LEN];
};
BPF_PERF_OUTPUT(events);

int hello(struct pt_regs *ctx) {
    struct data_t data = {};

    data.pid = bpf_get_current_pid_tgid();
    data.ts = bpf_ktime_get_ns();
    bpf_get_current_comm(&data.comm, sizeof(data.comm));

    events.perf_submit(ctx, &data, sizeof(data));

    return 0;
}
"""
```

```
@bpf
@struct
class data_t:
    pid: c_int64
    ts: c_int64
    comm: str(16)  # type: ignore [valid-type]


@bpf
@map
def events() -> PerfEventArray:
    return PerfEventArray(key_size=c_int64, value_size=c_int64)


@bpf
@section("tracepoint/syscalls/sys_enter_clone")
def hello(ctx: c_void_p) -> c_int64:
    dataobj = data_t()
    dataobj.pid, dataobj.ts = pid(), ktime()
    comm(dataobj.comm)
    events.output(dataobj)
    return 0  # type: ignore [return-value]
```

Program structure

- https://github.com/pythonbpf/Python-BPF/blob/master/BCC-Examples/hello_perf_output.py

```python
from pythonbpf import bpf, map, struct, section, bpfglobal, BPF
from pythonbpf.helper import ktime, pid, comm
from pythonbpf.maps import PerfEventArray
from ctypes import c_void_p, c_int64


@bpf
@struct
class data_t:
    pid: c_int64
    ts: c_int64
    comm: str(16)  # type: ignore [valid-type]


@bpf
@map
def events() -> PerfEventArray:
    return PerfEventArray(key_size=c_int64, value_size=c_int64)


@bpf
@section("tracepoint/syscalls/sys_enter_clone")
def hello(ctx: c_void_p) -> c_int64:
    dataobj = data_t()
    dataobj.pid, dataobj.ts = pid(), ktime()
    comm(dataobj.comm)
    events.output(dataobj)
    return 0  # type: ignore [return-value]


@bpf
@bpfglobal
def LICENSE() -> str:
    return "GPL"
```

Program structure

- https://github.com/pythonbpf/Python-BPF/blob/master/BCC-Examples/hello_perf_output.py

```
def LICENSE() -> str:
    return "GPL"


# Compile and load
b = BPF()
b.load()
b.attach_all()

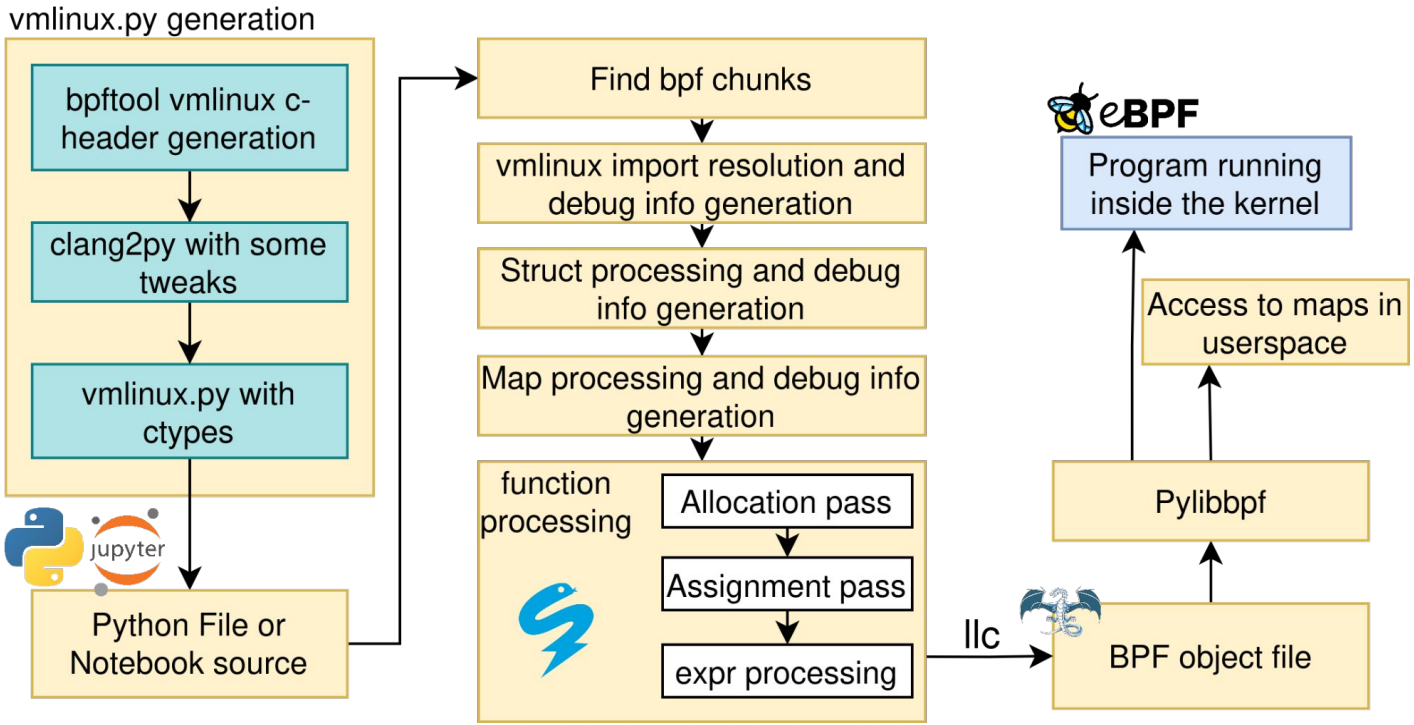
start = 0


def callback(cpu, event):
    global start
    if start == 0:
        start = event.ts
    ts = (event.ts - start) / 1e9
    print(f"[CPU {cpu}] PID: {event.pid}, TS: {ts}, COMM: {event.comm.decode()}")

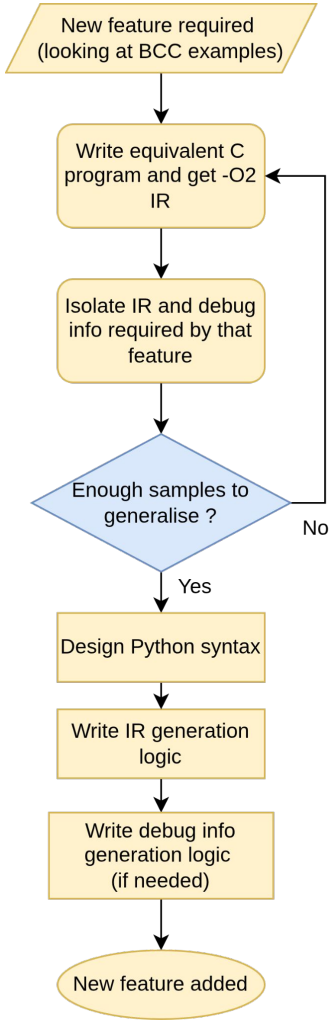

perf = b["events"].open_perf_buffer(callback, struct_name="data_t")
print("Starting to poll... (Ctrl+C to stop)")
print("Try running: fork() or clone() system calls to trigger events")

try:
    while True:
        b["events"].poll(1000)
except KeyboardInterrupt:
    print("Stopping...")
```

Under the hood



Development workflow



Demo time!

- You can try out the various BCC Tutorial examples ported over to Python-BPF (and their ipynb variants at):
  - https://github.com/pythonbpf/Python-BPF/tree/master#try-it-out

Why use Python-BPF?

- Abstract away typing - users must not have to think about typing unless absolutely unavoidable
- Abstract away verification details - try to mitigate most verification issues under the hood
- Allows existing Python devtools (LSPs, tree-sitters, linters) to be useful.
- Popular use-cases: enhanced data analysis and visualisation
- Can be used just to spit out the BPF object file, or load and attach as well.

To sum it up

- **It fights the verifier for you** (as much as it can).
- **It is much easier and less verbose than its alternatives.**
- **It is as extensible (if not more) as its alternatives.**

The big challenges – typing and vmlinux

- The biggest challenge till now, trying to make a loosely typed frontend work
- Python does not have pointers - required by helpers and kfuncs!
- Memory allocations in the first basic block
- Constants, enums and simple vmlinux structs (The ones with integer arguments and structs) work successfully as well as casting to different struct types from int64s in the fields of these structs.
- These structs are also CO-RE enabled as we generate DWARF debug info too.

Current state

- Some examples ported from BCC (with enhancements): https://github.com/pythonbpf/Python-BPF/blob/master/BCC-Examples
- Support for BPF Maps: Hash, PerfEventArray, RingBuf
- Support for Helpers: https://github.com/pythonbpf/Python-BPF/issues/63
- Work under way for kfuncs and porting more BCC examples.

Moving forward

- Better error handling and reporting (high)
- Documentation, quick start guides (high)
- Improving vmlinux.py support
- Better type tracking to reduce stack allocations

Our goal

Short term:



Long term:

Make Python-BPF one of the most popular choices for writing eBPF

WE WANT YOU!

- You can help us achieve our goal
- Share your wishlist, and if you / your org will have a formal interest in this project
- Planning a soft release @ FOSDEM 2026



Questions?

- https://github.com/pythonbpf/Python-BPF
- Open issues for RFCs, or reach out to us!
- pragyanshchaturvedi18<at>gmail<dot>com, varunrmallya<at>gmail<dot>com